The program was run on an Apple M5 Pro with 18 cores (6 performance cores and 12 efficiency cores). From 1 to 8 threads, the normalized execution time stays close to 1 (between 0.99 and 1.15), so adding threads gave no real speedup, because thresholding a 512x512 image is very little work and the whole program finishes in about 0.2 ms with a single thread. Starting at 16 threads the time goes up, and from 64 threads on it roughly doubles every time the thread count doubles, reaching 25.62 times the single-thread time at 512 threads. This happens because every thread has a fixed cost to create and join, and that cost grows with the number of threads while the total amount of work stays the same. Once the thread count goes past the 18 available cores, threads also have to share cores and the OS has to switch between them, which adds more overhead without making the work finish any faster.

Thread Count	Execution Time (Sec)				Normalized Execution Time
	run1	run2	run3	average	
1	0.000236	0.000214	0.000199	0.000216333333333333	1
2	0.000233	0.000196	0.000215	0.000214666666666667	0.99
4	0.000185	0.000404	0.000156	0.000248333333333333	1.15
8	0.000188	0.000318	0.00019	0.000232	1.07
16	0.000291	0.000254	0.000626	0.000390333333333333	1.8
32	0.000426	0.000761	0.000395	0.000527333333333333	2.44
64	0.000995	0.000687	0.000697	0.000793	3.67
128	0.001376	0.001375	0.001683	0.001478	6.83
256	0.002735	0.002668	0.002704	0.00270233333333333	12.49
512	0.005581	0.005545	0.005504	0.00554333333333333	25.62

